SEASONAL STYLES HUB

A Secure Serverless MultiPage E-Commerce Platform

PRESENTED BY

POTHINA PAVAN KALYAN
PULI ZENESWARI
OTURI LAKSHMI HANEELA
PALAPARTHI RAMYA
PULIPAKA SRINIVASRAO

PROJECT OBJECTIVES & CHALLENGES

1. The Modern E-Commerce Standard Dynamic Catalogs
2. Security & Authentication Requirements
3. Persistent Session Management
4. Cost & Maintenance Overhead

WHY WE BUILT THIS

Raising the shopper experience bar

- Customers expect fast browsing, trustworthy login, and frictionless navigation.
- They also expect their cart to stay intact across devices and sessions.

Limits of traditional deployments

- Always-on servers increase upkeep effort and slow down iteration cycles.
- Idle infrastructure creates waste when traffic is low or seasonal.

One solution, three outcomes

- The platform improves usability, strengthens security, and reduces operations overhead.

PROBLEM STATEMENT

- Traditional E-Commerce Applications require expensive server maintenance
- Managing user authentication using custom database is insecure
- Shopping carts are often not synchronized across multiple devices
- Scaling during high customer traffic that increases the infrastructure costs

SOLUTION OVERVIEW

- A Cloud-native server-less shopping platform built using AWS.
- Static Web Pages are hosted securely on Amazon S3.
- Products are filtered automatically base on selected seasons.
- Shopping cart data is securely stored in Amazon DynamoDB.
- AWS Lambda handles all backend business logic.
- Amazon API Gateway connect the frontend with backend services.

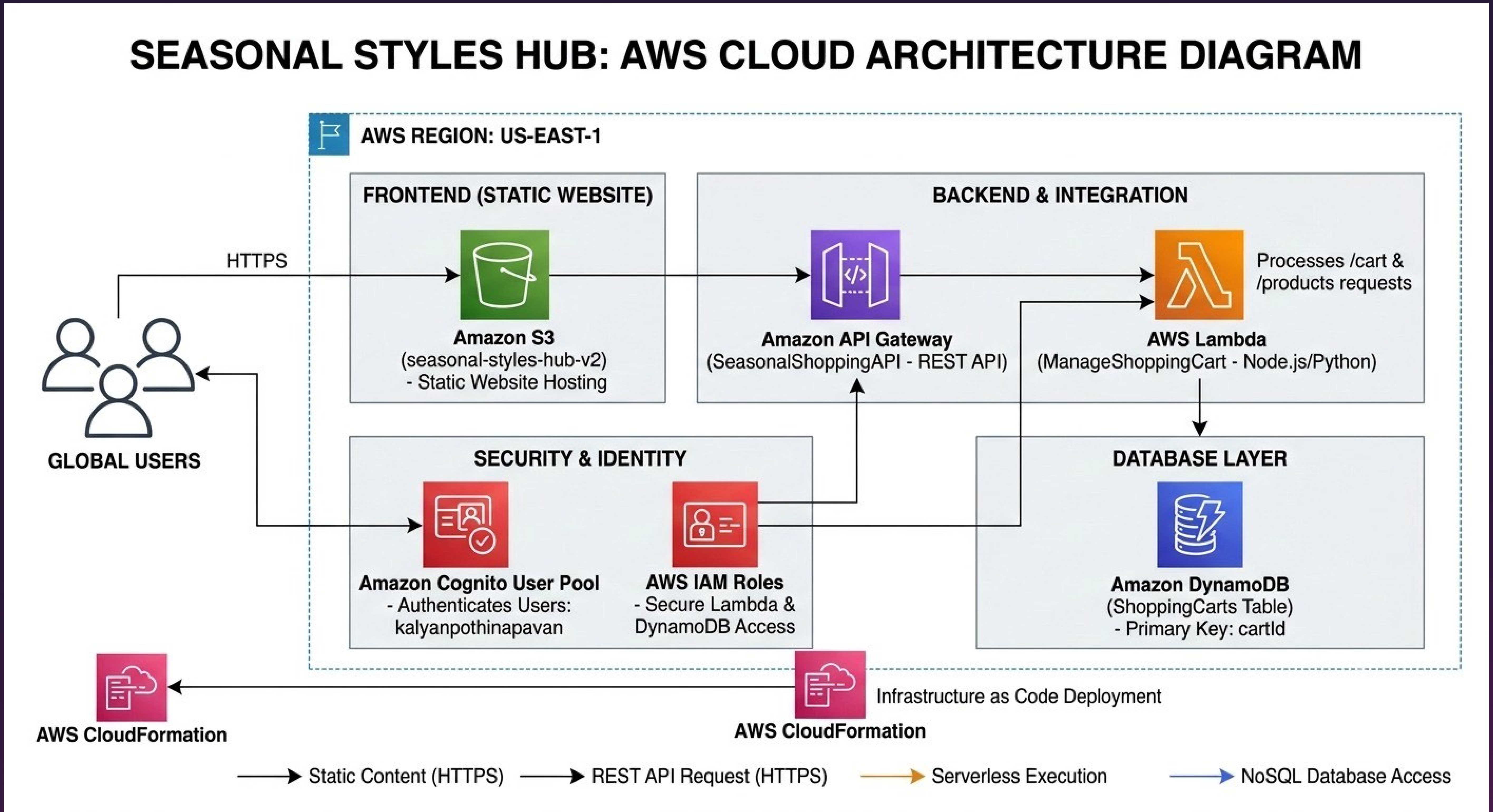
KEY FEATURES & TECHNICAL IMPLEMENTATIONS

- Secure Authentication
- Dynamic Seasonal Product Filtering
- Persistent Shopping cart across devices
- Responsive multi-page user Interface
- fast Serverless backend using AWS Lambda
- Secure NoSQL data storage using DynamoDB

AWS SERVICES USED

AWS Services	Purpose
Amazon S3	Hosts Static Website files
Amazon Cognito	User Authentication & Authorization
API GateWay	Routes API Requests
AWS Lambda	Executes backend logic
DynamoDB	Stores products & shopping carts

ARCHITECTURE DIAGRAM



FRONTEND EXPERIENCE

Multi-page storefront

Customers navigate Home, Cart, Login, About, and Contact pages smoothly.

Season-based browsing

A season dropdown refreshes the catalog instantly without page reloads.

Modern, responsive UI

Clean layouts, hover effects, and responsive scaling improve shopping usability.

DATA MODEL DESIGN

Catalog data table

- SeasonalProducts stores item details for fast season-based listing.
- Each record supports consistent rendering in the product grid UI.

Cart data table

- ShoppingCarts uses email as Partition Key for user isolation.
- productId as Sort Key enables multiple items per cart.

What this design enables

- Quick lookups for saved carts across logins and devices.
- Clean separation between catalog data and user-specific state.

FUTURE ENHANCEMENTS

Checkout completion

- Add a payment gateway to close the loop from cart to purchase.
- Keep the flow consistent with the current secure login experience.

Weather-driven personalization

- Auto-switch themes and featured collections using live weather signals.
- Make the store feel local by matching products to conditions.

Admin analytics portal

- Provide a secured dashboard for inventory updates and collection health.
- Track buying patterns to guide merchandising and seasonal planning.

KEY TAKEAWAYS

Operational simplicity

- Fully serverless AWS setup minimizes maintenance and eliminates idle infrastructure.
- Frontend and backend remain decoupled, enabling safe, independent updates.

Security by design

- Amazon Cognito provides **enterprise-grade identity** without custom credential storage.
- Token-based access keeps user sessions controlled and consistently validated.

Scalable foundation

- API Gateway, Lambda, and DynamoDB scale automatically with shopping demand spikes.
- A practical base to extend into payments, weather themes, and admin insights.